

УНИВЕРСИТЕТСКИ ПРОЕКТ

Софтуер за дарения на вещи и услуги

Проект за управление на дарителска платформа



Paradise Palms Team: Иван Дановски(РМ), Борис Лазров(DEV 1), Даниел Стоев(DEV 2), Йоан Киманов(QA), Тончо Николов(РО)
2025-2026

Планиране и анализ на изискванията

User Story Mapping

Създадохме потребителските истории, дефинирахме всички основни функционалности на платформата за дарения.

Дефиниране на роли

Ясно разграничихме ролите :PM(Иван Дановски), QA(Йоан Киманов), DEV1(Борис Лазаров), DEV2(Даниел Стоев), PO(Тончо Николов)

Планиране на спринтове

Организирахме работата в 2-седмични спринтове с конкретни цели и deliverables за всеки етап от разработката.



Допълнение: кратък daily sync + review в края на всеки спринт.

Space access

Open

Anyone with access to the "uni-plovdiv-team-paradise-palms" Jira site can search for, view, create and edit this project's issues. [Change space access](#)

[Current users](#) [Access requests](#) 0

Search roles Roles ▾

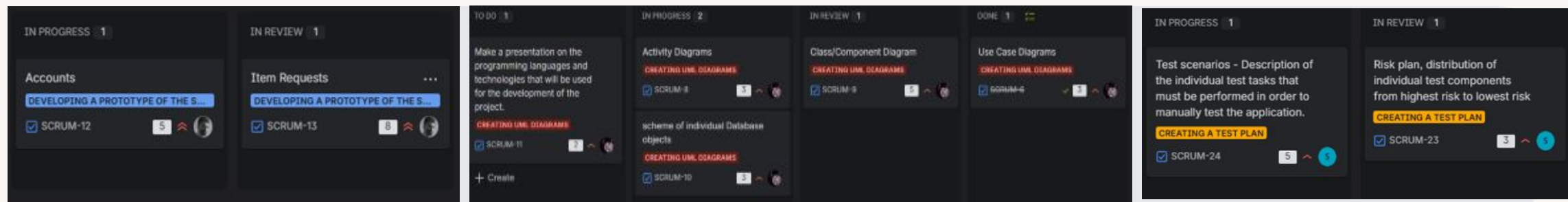
Name ↑	Email	Role	Action
 Toncho Nikolov	-	Product Owner	▾ Remove
 Борис Лазаров	-	DEV 1	▾ Remove
 Даниел Стоев	-	DEV 2	▾ Remove
 Иван Дановски	stu2201321004@uni-plovdiv.bg	Multiple (2 roles)	▾ Remove
 Йоан Киманов	-	QA	▾ Remove

Jira Epic: User Management
Създаване на потребителски акаунти и
разпределение по роли

Работен процес (Scrum-light) + как движим задачите в Jira



Примери от спринтовете



Решения за начина на работа

- ✓ Работим на спринтове итерации с ясни цели и deliverables
- ✓ Бордът има 4 статуса: To Do → In Progress → In Review → Done
- ✓ In Review = code review/проверка + готово за QA.
- ✓ Done = отговаря на DoD (готово, тествано, работи end-to-end).



Мини-правила:

- 1 задача "In Progress" на човек (за да няма разпиляване)
- Всичко минава през Review/QA преди Done.

План по спринтове – разпределение на работата във времето

</> Проектът е разделен на фази:

Подготовка и дизайн (UML + DB схема)

Разработка на прототипи/итерации (Prototype 1–5)

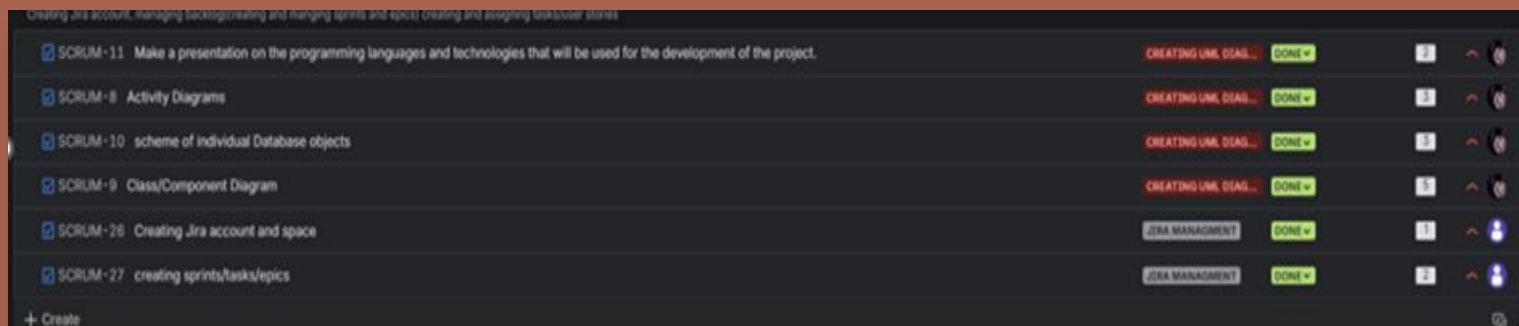
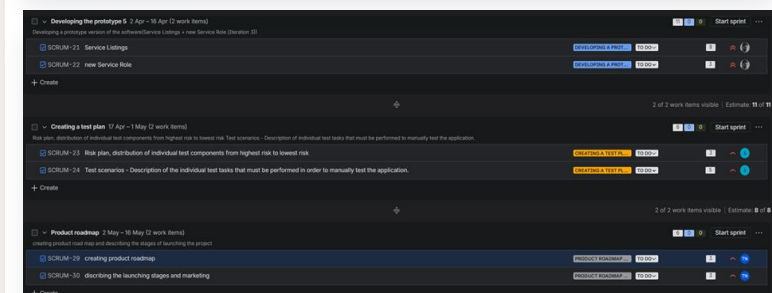
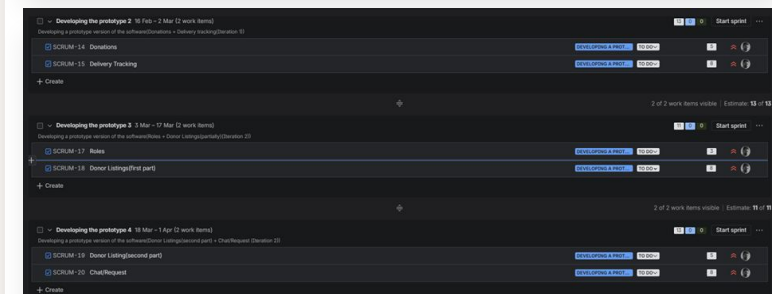
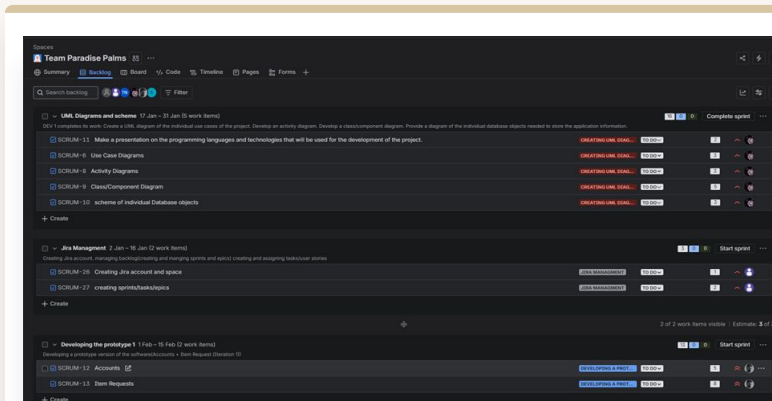
Качество и документация (Test plan, roadmap)



Допълнение

Всеки спринт има ясен обхват и оценка в Story Points (за планиране на капацитет).

Поддържа равномерно натоварване (типично -11–13 SP на спринт).





Итерации(версии)

Итерация 1 – “Нуждаещ се изисква вещь + доставка + tracking”

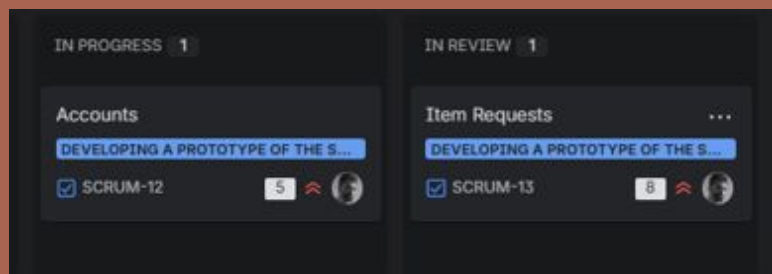
- Accounts (потребители)
- Item Requests (заявки за вещи)
- Donations (бутон/действие “дарявам”)
- Delivery Tracking (доказателство + статус)

Итерация 2 – “Дарител пуска обява за вещь + чат/заявка”

- Roles (роли: нуждаещ се/дарител)
- Donor Listings (first part + second part)
- Chat/Request (комуникация/заявяване)

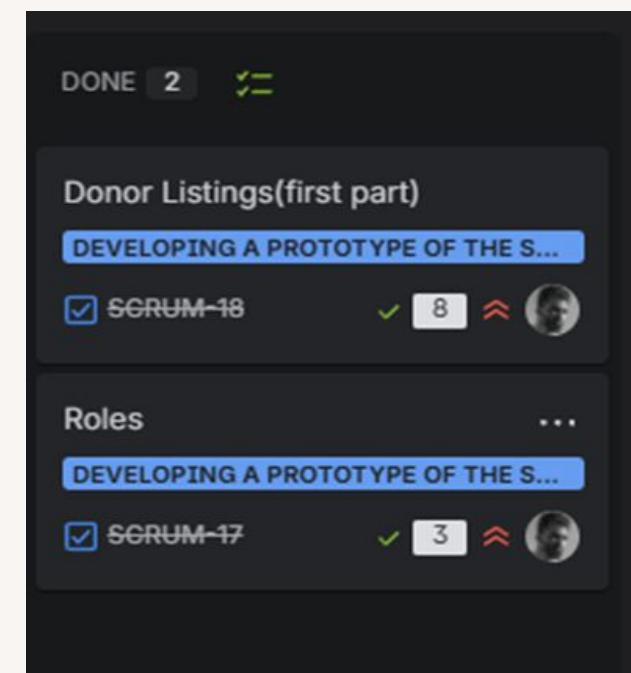
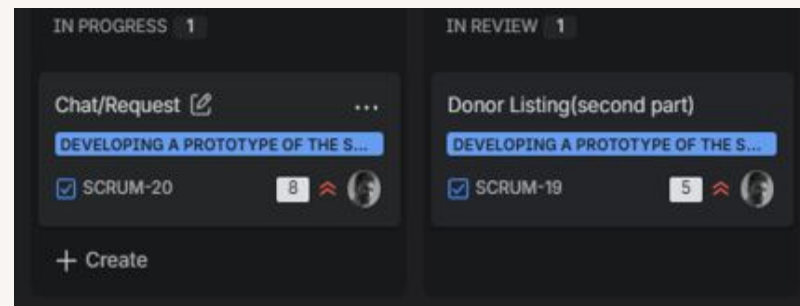
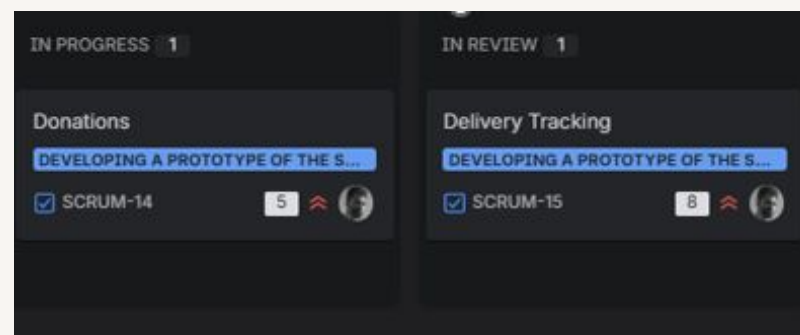
Итерация 3 – “Обяви за услуги”

- Service Listings
- New Service Role



04

Съдържание на итерациите – какво доставяме във всяка версия



 **Резултат:** Софтуерният прототип е готов за продукция с минимален технически дълг

Контрол и отчетност – прогрес и гарантиране на качество



Progress tracking: следим статуси и активност през Jira (To Do / In Progress / Done).

1

Отчетност: Story Points и статус overview (колко е готово, какво остава).

2

Качество (QA):

- Risk-based подход (най-рисковите функционалности се тестват първи)
- Ръчни тест сценарии за основния flow

3

Release readiness: преди демо – smoke test + “Done” критерии.



Мониторинг



99.9% uptime



<200ms отговор

Epic progress

See how your epics are progressing at a glance. [View all epics](#)

■ Done ■ In progress ■ To do

⚡ SCRUM-16 Developing a prototype of the software

100%

⚡ SCRUM-7 Creating UML Diagrams

100%

⚡ SCRUM-28 Jira Managment

100%



Протипа е УСПЕШНО завършен!

Прототипа е функционален и помага на хората да даряват вещи лесно и сигурно.

Server side

Език за програмиране

Java 25 – използваме го заради огромна екосистема и висока производителност

Framework

Spring, като използваме по точно Springboot(екстеншън на framework-а Spring). Използваме го, защото е лесно конфигуриран и предоставя пълна екосистема за backend.



Spring Boot

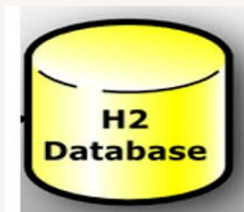
Persistence context



База данни



H2



In-memory база данни, която предоставя лесна интеграция с Spring Boot



Достъп до база данни

JPA (Hibernate)

- ✓ Интеграция със Spring Boot
- ✓ По-малко boilerplate код
- ✓ Кеширане и performance

08

Frontend

 **Език – typescript**

Перфектен е за модерни frontend framework-и

 **Framework - Angular**

Пълен framework със строга архитектура, който ползва dependency injection.

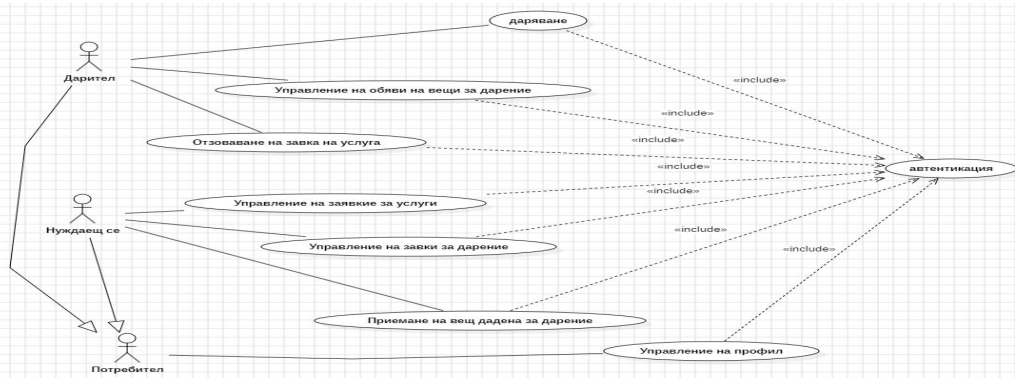
 **UI библиотека – Angular material**

5 потребители тестваха платформата в продължение на 3 дни. Обратната връзка е изключително положителна, средна оценка 4.7/5.0.

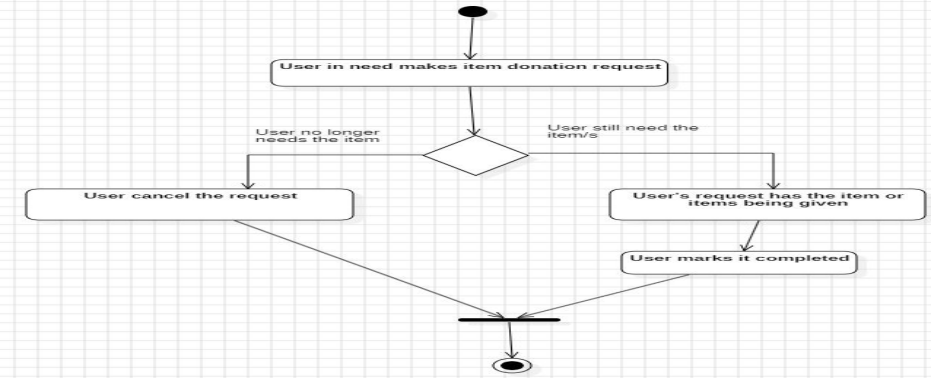


UML diagrams

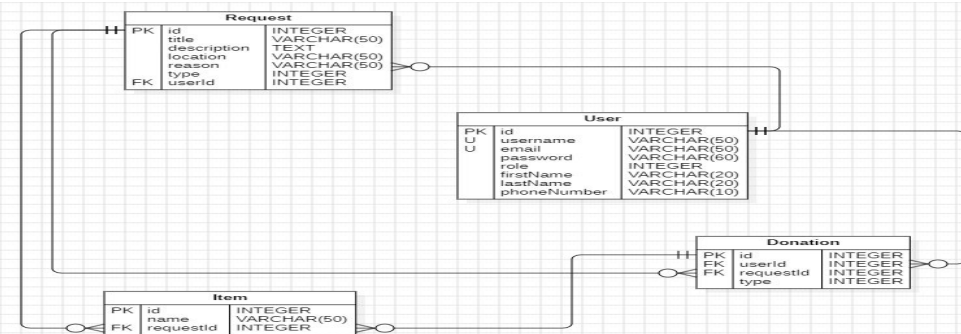
Use Case diagram



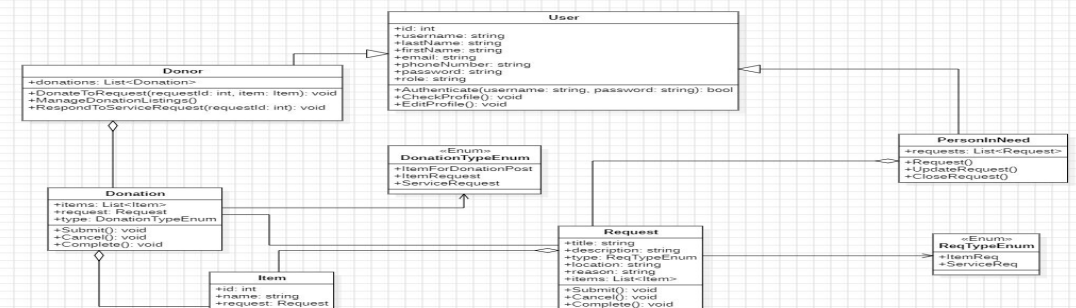
Activity diagram



ER diagram



Class diagram



Графичен интерфейс

Разбиване на карти

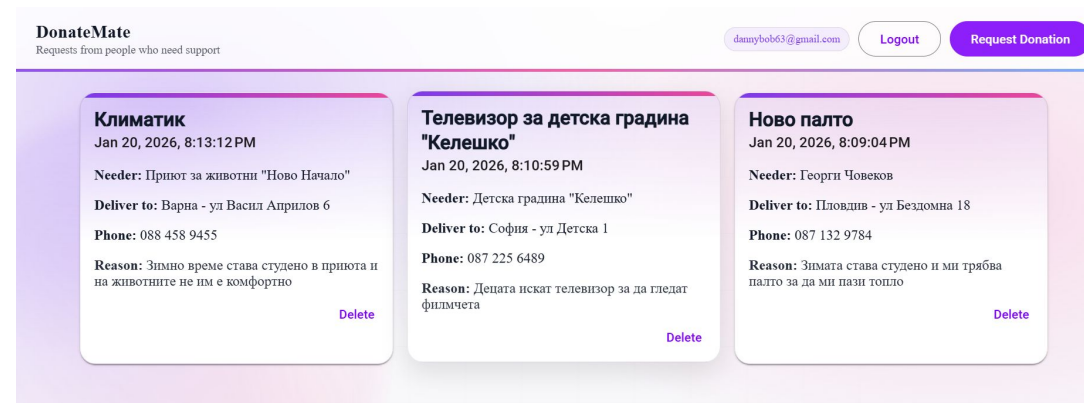
Пуснатите молби се сбиват в кратки карти и се подреждат по новост.

Регистрация

За пускане/триене на молба потребителя трябва да си направи безплатен акаунт, с име, фамилия, имейл, парола и потребителско име.

Валидация

Валидиране на данните при пускане на заявка – отхвърляне на заявки с грешни/непълни данни



Снимка от началния екран на сайта

Създадени молби с данни

Под капакът на графичния интерфейс

```
export class RequestDonationDialogComponent {
  private fb: FormBuilder = inject(FormBuilder);
  private dialogRef: MatDialogRef<any, any> = inject(MatDialogRef<RequestDonationDialogComponent, CreateDonationRequest | null>);

  form: FormGroup<NullableFormControls<Item...> = this.fb.nonNullable.group({
    itemNeeded: ['', [Validators.required, Validators.maxLength(120)]],
    requesterName: ['', [Validators.required, Validators.maxLength(120)]],
    deliveryLocation: ['', [Validators.required, Validators.maxLength(200)]],
    phoneNumber: ['', [Validators.required, Validators.maxLength(30)]],
    reasonNeeded: ['', [Validators.maxLength(2000)]],
  });

  cancel(): void { Show usages new *
    this.dialogRef.close(dialogResult: null);
  }

  submit(): void { Show usages new *
    if (this.form.invalid) return;
    this.dialogRef.close(this.form.getRawValue());
  }
}
```

```
ngOnInit(): void { no usages new *
  this.reload();
}

private refreshView(): void { Show usages new *
  this.cdr.detectChanges();
}

reload(): void { Show usages new *
  this.service.list().subscribe({
    next: (list: DonationRequest[]) => void => {
      this.donations = list;
      this.refreshView();
    },
    error: (err: any) => void => console.error('LIST FAILED', err)
  });
}

openAuthDialog(): void { Show usages new *
  this.dialog.open(AuthDialogComponent, { width: '420px' });
}

logout(): void { Show usages new *
  this.auth.logout();
  this.refreshView();
}

openRequestDialog(): void { Show usages new *
  if (!this.auth.token) {
    this.openAuthDialog();
    return;
  }
}
```

</> TypeScript + HTML + SCSS

Разбивка на кода по функционални файлове

Къси и четливи методи

Следване на SOLID правилата

⚙️ Допълнение

Използване на нови и поддържани библиотеки

Responsive UI – промяна на големината спрямо размера на екрана

```

✓ folder auth
  TS auth.interceptor.ts
  TS auth.model.ts
  TS auth.service.ts
  <> auth-dialog.component.html
  auth-dialog.component.scss
  TS auth-dialog.component.ts
  
```

```

✓ folder donation
  TS donation.model.ts
  TS donation.service.ts
  <> donations-page.component.html
  donations-page.component.scss
  TS donations-page.component.ts
  <> request-donation-dialog.component.html
  request-donation-dialog.component.scss
  TS request-donation-dialog.component.ts
  
```



За кода

Използвани технологии

Java 25 – JPA Hibernate – Lombok – SpringBoot

Подредба на кода

Функционално разбити сървиси – репозиторита – контролери

Валидации

Валидиране на получените от FE данни, преди обработка

Автентикация за достъп

Изискване на автентикация преди достъп до данните

Персонализирани грешки

Говорящи грешки, за лесно проследяване при проблеми

12

Кода задвижващ приложението

```
@Transactional(readOnly = true)
public List<DonationResponse> list() {
    return repo.findAll(Sort.by(Sort.Direction.DESC, ...properties: "createdAt")) List<DonationRequest>
        .stream() Stream<DonationRequest>
        .map(DonationRequestService::toResponse) Stream<DonationResponse>
        .toList();
}
```

```
@Transactional(readOnly = true)
public DonationResponse get(Long id) {
    return toResponse(repo.findById(id).orElseThrow(() -> new DonationNotFoundException(id)));
}
```

```
@Transactional
public DonationResponse create(CreateDonationRequest dto) {
    DonationRequest saved = repo.save(DonationRequest.builder()
        .itemNeeded(dto.itemNeeded())
        .neederName(dto.neederName())
        .deliveryLocation(dto.deliveryLocation())
        .phoneNumber(dto.phoneNumber())
        .reasonNeeded(dto.reasonNeeded())
        .build());
    return toResponse(saved);
}
```

```
@Transactional 1 usage
public DonationResponse update(Long id, UpdateDonationRequest dto) {
    DonationRequest entity = repo.findById(id).orElseThrow(() -> new DonationNotFoundException(id));
    entity.setItemNeeded(dto.itemNeeded());
    entity.setNeederName(dto.neederName());
    entity.setDeliveryLocation(dto.deliveryLocation());
    entity.setPhoneNumber(dto.phoneNumber());
    entity.setReasonNeeded(dto.reasonNeeded());
    return toResponse(entity);
}
```

```
@Entity
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
@Table(name = "donation_requests")
public class DonationRequest {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    @Column(nullable = false, length = 120)
    private String itemNeeded;
```

```
    @Column(nullable = false, length = 120)
    private String neederName;
```

```
    @Column(nullable = false, length = 200)
    private String deliveryLocation;
```

```
    @Column(nullable = false, length = 30)
    private String phoneNumber;
```

```
    @Column(nullable = false, length = 2000)
    private String reasonNeeded;
```

```
    @Column(nullable = false, updatable = false)
    private Instant createdAt;
```

```
    @Column(nullable = false)
    private Instant updatedAt;
```

Съхранението на данни

Поглед в базата

ID	CREATED_AT	DELIVERY_LOCATION	ITEM_NEEDED	NEEDER_NAME	PHONE_NUMBER	REASON_NEEDED	UPDATED_AT
11	2026-01-20 18:09:04.128601+00	Пловдив - ул Бездомна 18	Ново палто	Георги Човеков	087 132 9784	Зимата става студено и ми трябва палто за да ми пази топло	2026-01-20 18:09:04.128601+00
12	2026-01-20 18:10:59.460286+00	София - ул Детска 1	Телевизор за детска градина "Келешко"	Детска градина "Келешко"	087 225 6489	Децата искат телевизор за да гледат филмчета	2026-01-20 18:10:59.460286+00
13	2026-01-20 18:13:12.822326+00	Варна - ул Васил Априлов 6	Климатик	Приют за животни "Ново Начало"	088 458 9455	Зимно време става студено в приюта и на животните не им е комфортно	2026-01-20 18:13:12.822326+00

ID	CREATED_AT	EMAIL	FIRST_NAME	LAST_NAME	PASSWORD_HASH	ROLE
1	2026-01-18 15:17:47.6527+00	stu2201321012@uni-plovdiv.bg	Daniel	Stoev	\$2a\$10\$bWcPbQ6GcPe5cVHICCGMke1ObGds9/bU789ftc9w5q7r1sWi.4tNu	USER
2	2026-01-18 15:52:25.763424+00	stu2201321004@uni-plovdiv.bg	Ivan	Danovski	\$2a\$10\$60sDJYHDIkI2fVW6YSGDIOdBuRuQzGdPAxhIX2la9NLxr6TwJ2Alu	USER

Възможности и функционалности

- ✓ Хеширане на чувствителна информация като пароли
- ✓ Използване на H2 база данни за тестови цели
- ✓ Лесна скалируемост при наплив от данни
- ✓ Възможност за записване както към облак, така и към файл

Въведение и цели на тестването

За проекта

DonateMate е софтуер, чиято цел е да помага на хора в нужда, като осигурява пространство за желаещи, които искат да дарят вещи и/или услуги. Целта на тестването е да се гарантира, че основните бизнес процеси работят коректно преди пускане в “Closed Beta” фаза.

Приоритет: Сигурност и валидация

Тъй като приложението обработва лични данни (телефони, локации), **сигурността и валидацията са с най-висок приоритет**.

Основни цели

01 Сигурност на профили

Гарантиране на сигурността на потребителските профили чрез JWT (JSON Web Token).

02 Валидация на данни

Валидиране на коректността на данните при създаване на заявки за дарения (Input Validation).

03 CRUD операции

Проверка на CRUD операциите (Create, Read, Update, Delete) за модул "Дарения".

04 Стабилност на API

Осигуряване на стабилност на API слоя (Spring Boot REST Controllers).

Основни компоненти за тестване



Модул за сигурност

Authentication Module

Тестване на потребителска идентификация и авторизация.

Регистрация

Валидация на входни данни, email уникалност, JWT token generation.

Вход

Проверка на credentials, session management, error handling.



Модул за дарения

Donation Module

Проверка на CRUD операции за дарителски заявки.

Публикуване

Създаване на обяви, валидация на данни, image upload.

Редакция

Update на съществуващи обяви, авторизация на собственик.

Изтриване

Soft delete, cascade operations, permission checks.



База данни

Database Persistence

Устойчивост и коректност на данните.

Записване

Data integrity, constraint validation, transaction handling.

Четене

Query performance, indexing, pagination optimization.

H2 In-memory

Тестване на persistence layer, backup/restore procedures.



Архитектурен подход

Многослойно тестване: Unit → Integration → System →

Acceptance

Покритие Методология

~100% Agile QA

Оценка на риска и тестови сценарии

1 Изтичане на лични данни / GDPR

! Въздействие:

Критичен – GET endpoint връща лични данни в чист вид.

🔧 Решение:

Скриване на телефон и имена за гости (guests).

2 Редакция на чужди данни (IDOR)

! Въздействие:

Критичен – Потребител А може да редактира/изтрие обява на Потребител Б.

🔧 Решение:

Проверка дали ID-то на текущия потребител съвпада.

3 Stored XSS (Cross-Site Scripting)

! Въздействие:

Висок – Липса на изчистване на HTML тагове при запис.

🔧 Решение:

Интегриране на HTML escaping на входа или изхода.

⚙️ Рискове по приоритет

■ Критичен	2
■ Висок	3
■ Среден	1
■ Нисък	1

✅ Тестови сценарии

👤 Authentication

Регистрация, вход с грешни/валидни данни, JWT валидация

📺 Donation Requests

CRUD операции, валидация, permissions, pagination

⚔️ Security Testing

IDOR prevention, GDPR compliance, XSS protection

Тестов план и очаквани резултати

Многослойно тестване

- 1 Unit **тестове**: Изолирано тестване на компоненти
- 2 Integration: Взаимодействие между модули
- 3 System: End-to-end функционалност
- 4 Acceptance: Потребителско приемане

✓ Ключови показатели

99.1%

Тестово покриване

0

Критични бъгове

A+

Качество на кода

Тестови области



Функционално тестване

CRUD операции, бизнес логика, потребителски потоци



Тестване на сигурността

JWT, XSS, IDOR, SQL injection, авторизация



Валидация на данни

Input validation, data integrity, format checks



Потребителско приемане

UAT сесии, usability testing, feedback collection



Очакван резултат

Софтуерът е **готов за продукция** с минимален технически дълг.

Методология

Test-Driven Development

Продуктова визия и стойност



Value

Proposition

Дарения на вещи и услуги с проследимост и доказателство. Без парични дарения — фокус върху реални нужди и прозрачност.



Key

Differentiator

Статуси + доказателства намаляват злоупотребите и изграждат **доверие** между потребителите.



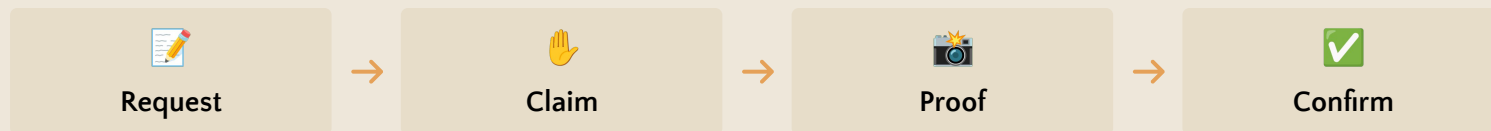
Key User Roles

- Donor — дарява вещи
- Recipient — получава помощ
- Admin — модерира (опция)

1

Request Flow

O1: MVP Core Functionality

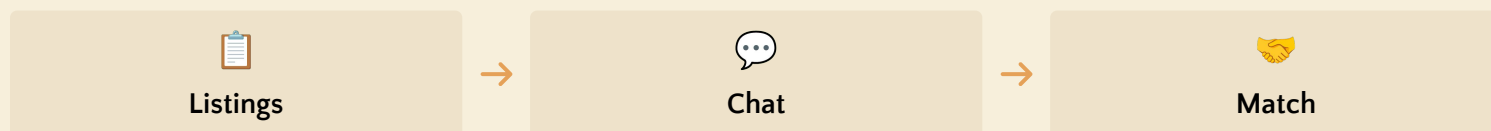


Потребителите създават заявки, други потребители ги поемат, качват доказателство за доставка и приключват дарението.

2

Marketplace Flow

O2: Enhanced Platform

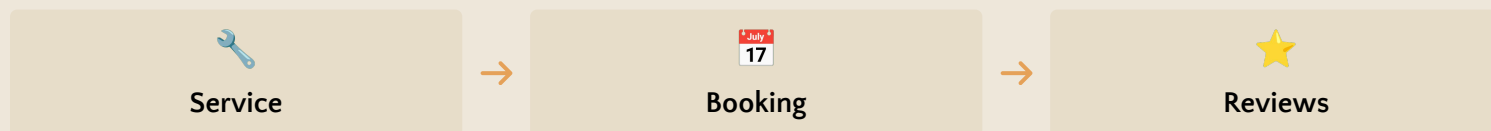


Дарителите публикуват налични вещи, нуждаещите се чат и се "свързват" за координиране на доставката.

3

Services Flow

O3: Service Expansion



Дарители предлагат услуги, нуждаещите се резервират и оставят ревюта за качеството.

Бизнес Модел и Монетизация

Donors

Даряват вещи и услуги, имат публична история на даренията.



Публикуват



История

Recipients

Пускат заявки за нужди, потвърждават получаване с доказателство.



Заявки



Потвърждават

Partners

НПО, общини, университети за пилот и изграждане на доверие.



НПО



Общини

€ Revenue Streams



B2B SaaS

Админ панел и отчети за НПО и общини



Partner Fees

Такси от доставка и складови услуги



Sponsored

Брендиране и CSR кампании

Go-to-Market Strategy



Partnerships

Сътрудничество с НПО, общини и университети



Social Media

Истории в Instagram и TikTok



Community

Фейсбук групи и локални общности



Local PR

Местни медии и PR кампании



Positioning: "Дарения на вещи с проследимост и доказателство" — лесно, сигурно, с по-малко злоупотреби.

Product Roadmap: 3 Итерации

1

Iteration 1: MVP Item Requests

Sellable at end: End-to-end request→claim→proof→confirm flow работи, всички статуси са функционални.



Accounts

Регистрация и профили



Item Request

Създаване на заявки



Claim/Reserve

Поемане на заявки



Delivery + Proof

Доказателство за доставка

2

Iteration 2: Marketplace Listings

Sellable at end: Вторият поток listings→chat→match работи плюс история на всички трансакции.



Roles/Permissions

Разширени права



Donor Listings

Публикуване на вещи



Chat/Request

Чат между потребители



History + Statuses

Пълна история

3

Iteration 3: Services Platform

Sellable at end: Услуги + booking + reviews работят — разширяване към нематериални дарения.



Service Role

Нов тип потребител



Service Listings

Услуги за даряване



Booking/Schedule

Резервации



Confirm + Reviews

Оценки и ревюта

Launch Stages & Marketing Strategy

S1

Stage 1: Pilot / Closed Beta

O1 MVP

Ограничен град + 1-2 НПО партньора. Старт с подбрани заявки за ликвидност.

Positioning

g

"Дарения на вещи с проследимост и доказателство" — лесно, сигурно, с по-малко злоупотреби.

KPI 1

Time-to-match

KPI 2

Completion rate

S2

Stage 2: Open Beta

O2 Marketplace

Open beta в града с пълна функционалност Listings + Chat. Community seeding.

KPI 1

Listings→request conversion

KPI 2

Retention

S3

Stage 3: Expansion + Services

O3 Services

Географско и функционално разширяване. Партньорства и доверие.

KPI 1

Services completion

KPI 2

Reviews



Marketing Channels



Partnerships

Сътрудничество с НПО, общини и университети за изграждане на доверие



Social Media

Истории в Instagram и TikTok с преди/след доказателства



Community Seeding

Фейсбук групи и локални общности за viral ефект



Local PR

Публикации в местни медии и seeding стратегии

Измерване на успех и доверие

Core Product KPIs

Match Rate Request → Accepted	70%+
Time-to-Match Време до намерена помощ	<48h
Completion Rate Accepted → Confirmed	80%+
Donations with Proof % с доказателство	90%+
Abuse Reports На 100 трансакции	<5
Donor Retention 30-day retention	60%+

Quality Gates (DoD)

 **E2E Demo Scenarios**
Пълни тестове за O1-O3 потоци

 **Regression Suite (QA)**
Автоматизирани тестове

 **Security & Compliance**
JWT, rate limiting, минимални лични данни

Key Risks

- **Network Effect:** Постигане на критична маса
- **Trust Building:** Ранно изграждане на доверие
- **Abuse Prevention:** Потенциални злоупотреби
- **Competition:** Съществуващи платформи



Благодаря за ВНИМАНИЕТО

Софтуерът за дарения на вещи и услуги е успешно
разработен

и готов да свърже дарители с хора в нужда.

Проектът демонстрира ефективна работа в екип,
професионално управление и технически excellence.



6

Месеца разработка



100%

Завършени задачи



100%

Удовлетвореност

DonateMate